CPLP - clase 1
sintáxis

Sintaxis → Conjunto de reglas para la escritura del programa

Semántica → Conjunto de reglas para dar significado a los programas sintácticamente válidos.

La sintáxis debe ayudar al programador a escribir programas correctos sintácticamente
↳ Establece reglas para la comunicación con el procesador.

Compuesta por:
- ↳ Alfabeto / Conjunto de caracteres
- ↳ Identificadores: Nombres definidos por el programador para vars/funcs
  - ↳ Letras y dígitos, debe comenzar por una letra
- ↳ Operadores: Suma (+), resta (-), etc.
- ↳ Comentarios: Hacen los programas más legibles
- ↳ Keywords: Palabras clave con un significado dentro de un contexto
- ↳ Palabra reservada: Keywords que No pueden usarse como identificadores.

Estructura semántica:
- ↳ words: Conjunto de chars y palabras para construir expresiones, sentencias, etc.
  - ↳ De constructos a partir del alfabeto, ej: if, while, for, etc.
- ↳ Expresiones: Funciones que a partir de un conjunto de datos, devuelve un resultado
- ↳ Sentencia: Instrucciones que usamos en el código.

Reglas léxicas: Define como se forma una word, ej: En PHP, las variables inician con $.

Reglas sintácticas: Define cómo formar las expresiones y sentencias: Cómo definir un if
- ↳ if(...) then
- ↳ if(...) {

Tipo de sintáxis:
- Abstracta: Estructura
- Concreta: Parte Léxica
- Pragmática: Uso Práctico

Define reglas mediante producciones

↳ <Dígito> ::= 0|1|2|3|4|5|6|7|8|9

No terminal ↲ ↳ Meta símbolo

Terminales

- Los <u>No</u> terminales son los que no se pueden seguir definiendo.

Gramática: $G = (N, T, S, P)$

N = Conjunto de símbolos no terminales

T = " Terminales

S = Símbolo distinguido de la gramática que pertenece a N

P = Producciones

Una gramática es ambigua si una sentencia puede derivarse de más de una forma.

$G_N = (N, T, S, P)$

N = { <Natural> <dígito> }

T = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

S = <Natural>

P = { <Natural> ::= <dígito> | <dígito> <Natural>,

<dígito> ::= 0|1|2|3|4|5|6|7|8|9 }

CPLP - Clase 2 - Semántica

Semántica: Describe el significado de los símbolos, palabras y frases de un lenguaje

Semántica estática: Se relaciona con las formas válidas (sintaxis)
↳ El análisis para el chequeo puede hacerse en compilación.

Gramática de atributo: A las construcciones del lenguaje se les asocia información a través de "atributos" asociados a los símbolos de la gramática
↳ Valor de una variable, tipo de una variable, lugar en memoria, etc
↳ Se enlazan a los terminales y no-terminales.

Reglas gramaticales	Reglas semánticas
decl → tipo lista-var	lista-var.at = tipo.at
tipo → INT	tipo.at = INT
tipo → FLOAT	tipo.at = FLOAT
lista-var → id	id.at = lista-var.at AñadeTipo(id.entrada, lista-var.at)
lista-var$_1$ → id, lista-var$_2$	id.at = lista-var.at AñadeTipo(id.entrada, lista-var.at) lista-var$_2$.at = lista-var.at

* Cursivas: no-terminal
* Imprenta: Terminal

Semántica dinámica: Aparece durante la ejecución de un programa
↳ Se aplica luego de comprobar que la sintaxis y que la semántica sea correcta
↳ Describe el proceso que una computadora sigue cuando se ejecuta un programa en un lenguaje en específico.

↳ Difícil de escribir, no existen herramientas estándar
↳ 2 soluciones formales complejas
↳ Semántica axiomática
↳ Semántica denotacional
↳ Semántica operacional

Axiomática: Trabaja con axiomas, ve al programa como una "máquina de estados"
↳ Estado: Predicado que describe los valores de las variables en ese estado → Estado anterior y a la ejecución del constructor
↳ Estado posterior
↳ Se describe con un predicado
↳ Cada sentencia se precede y contiene con una expresión lógica que describe las restricciones y relación entre los datos
↳ Precondición: $\{b != 0\}$
Postcondición: $\{a = b*c + r \wedge r < b\}$
Sentencia: Expresión que divide a por b

ej: a/b a | b
r c

Denotacional: Basado en funciones recursivas y modelos matemáticos
↳ Busco encontrar una función que pueda escribir la producción
↳ Define una correspondencia entre los constructores sintácticos y sus significados.

↳ <Nbin> ::= 0 | 1 | <Nbin>0 | <Nbin>1

→ Fnbin(0) = 0
↳ Fnbin(1) = 1

Fnbin(<Nbin>0) = 2* Fnbin(<Nbin>)
Fnbin(<Nbin>1) = 2* Fnbin(<Nbin>) + 1

Operacional → Los cambios de estado de la máquina cuando se ejecuta una sentencia del lenguaje de programación definen el significado de un programa.

↳ Este cambio de estado se da por un lenguaje de bajo nivel implementado sobre una máquina abstracta

Traducción de un lenguaje: Lenguaje de máquina

↳ Forma de dejar de programar en 1s y 0s (Lenguaje Assembler) y este lenguaje era convertido a binario mediante "Programa ensamblador".

↳ ◦ Traducción
◦ Compilación
◦ Traducción y compilación

Intérprete → Lee, analiza, decodifica y ejecuta uno a uno las sentencias de un programa
↳ Por cada acción posible hay un subprograma en lenguaje de máquina que ejecuta esa acción
↳ La interpretación es el llamado a estos subprogramas en el orden correcto

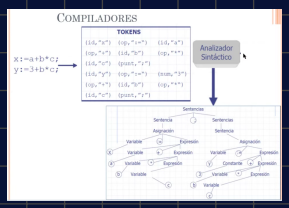
Compilación: Traduce un HLL a una versión en lenguaje de máquina antes de ser ejecutado.

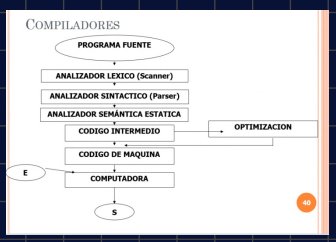
↳ HLL —Compilación→ Lenguaje objeto

↳ 2 etapas: Análisis → 1) Análisis léxico (Scanner): Valida las palabras y divide el programa en tokens (IDs, Key words, comentarios, delimitadores)
2) Análisis sintáctico (Parser): Valida las sentencias, usa los tokens
3) Análisis semántico (Semántica estática): Se procesan las estructuras sintácticas reconocidas, y toma forma la estructura del código.

Síntesis → 1) Optimización del código
2) Generación del código.

↳ Reconstruye el programa ejecutable, genera el código necesario y se optimiza el programa generado
↳ Uso la optimización optativa





CPLP - Clase 3 - Semántica de los lenguajes de programación

Entidades	Variables
Variable	Nombre Tipo Área de mem. etc
Rutina	Nombre Parámetro formal Parámetro reales, etc
Sentencia	Acción asociada

Descriptor: Lugar (Repositorio) donde se almacena la info de los atributos

Ligadura: Momento en el que un atributo se asocia con un valor

Binding time → estático: Antes de la ejecución y no se puede cambiar
→ dinámico: Durante la ejecución y puede cambiar
↳ Menos las constantes

Estabilidad de la ligadura: Si puede o no cambiar una ligadura una vez establecida

Momento de la ligadura estática:
- Definición del lenguaje
- Implementación del lenguaje
- Compilación (preprocesamiento)

Momento de la ligadura dinámica: Ejecución

Variable: Abstracción de una celda de memoria

L. Nombre: Identificador
- Tipo
- Alcance: Rango de instrucciones en el que se conoce el nombre, es visible, y puede ser referenciado
- L-valor: Lugar de memoria asociado con la variable, asociado al tiempo de vida
- N-valor: Valor codificado almacenado en la ubicación de la variable

Alcance estático: Define el alcance en términos de la estructura léxica del programa

Alcance dinámico: Define el alcance en tiempo de ejecución
↳ Confuso, siempre depende de "quién me llama"

- Espacio de nombres: Zona separada abstracta del código donde se pueden agrupar, declarar y definir objetos (vars, funciones, ID de tipos, clase, etc).
  ↳ Evita redundancia de nombres en proyectos grandes, usado en lenguajes de tipo dinámico.

- Antes de referenciar una variable debe ligársele un tipo
  ↳ Cuando a una variable se le asigna un tipo pasa a ser una instancia

- Fuertemente tipado: No se puede mezclar valores de tipo diferente
  ↳ Algunos lenguajes permiten aplicar conversiones.

- Tipos → Predefinidos: Descriptos en la definición del lenguaje (INT, char, etc)
  ↳ Definidos: Permite definir tipos a partir de los predefinidos y constructores
  ↳ TAD (Tipo Abstracto de Datos): Pilas, Listas, colas, Grafos, etc
    ↳ Estructura que representa a un nuevo tipo con un nombre que lo identifica

- Binding estático: El tipo se liga en compilación y NO puede ser cambiado
  ↳ Se establece mediante una declaración // Explícito
  ↳ Si no fue declarado, se deduce por reglas del lenguaje // Implícito
  ↳ El tipo se deduce de los tipos de sus componentes // Inferido
    ↳ Si no está definido el tipo, se infiere por reglas.

- Binding dinámico: El tipo se liga en tiempo de ejecución y puede cambiar.
  ↳ Mayor costo de implementación
  ↳ Chequeo dinámico
  ↳ Menor legibilidad

- **L-Value**: Área de memoria ligada a una variable. Las instrucciones de un programa acceden a la variable mediante su L-valor.
  - Tiempo de vida: Período de tiempo de la ligadura (vida de la variable (alocación y desalocación)).
  - Alocación: Momento en que se reserva la memoria para una variable
    - estática: Se hace en compilación, perdura hasta el fin del programa
    - Dinámica: Se hace en tiempo de ejecución
      - Automática: Cuando aparece una declaración al ejecutarse
      - Explícita: Requerido por el programador con la creación de una instancia (constructor) (ej. punteros)
    - Persistente: Existen en el entorno de ejecución. Persisten más allá de memoria (ej. archivos, Bases de datos)

- **R-Value**: Valor codificado almacenado en la locación asociada a la variable (L-valor)
  - Se interpreta de acuerdo al tipo de variable

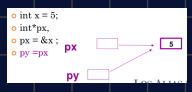
obj: (L-valor, r-valor) (dir., valor)

X := X + 1

- r-valor: Se accede a la variable por el L-valor, se puede modificar el r-valor (NO en constantes)
- L-valor

- **Variables anónimas**: Algunos lenguajes permiten que las vars. sin nombre sean accedidas por el r-valor de otra variable.
  - se denomina referencia o puntero

- **Alias**: 2 variables comparten un objeto si su ref. es la misma



- **Alias**
  distintos nombres → 1 entidad
- **Sobrecarga**
  1 nombre → distintas entidades

Sobrecarga: Se genera cuando un nombre referencia a más de una entidad
- Debe estar permitido por el lenguaje
- se requiere información para desambiguar

Dice: Se requiere información para desambiguar, gracias Gomez por tomar nota.

- Ej: Sobrecarga de constructores: Una entidad con distintos constructores de mismo nombre, se diferencian por sus parámetros (cantidad, orden, nombre, tipo).